Day 4

Friend Keyword in C++

- The friend keyword can be applied to **both functions** and **classes**.
- A **friend function** is a **non-member function** that is allowed to access the **private and protected members** of a class in which it is declared as friend.
- Normally, private members can only be accessed within the class, but using friend, we can give special access to specific functions or classes.

Friend Function

A **friend function** is declared inside a class using the keyword friend.
Although declared inside the class, it **is not a member function** of that class.

Syntax:

```
class ClassName
{
private:
    int data;
public:
    friend void functionName(ClassName&);     //Friend Function
};
```

Definition (outside the class):

```
void functionName(ClassName& ref)
{
    // Directly access private data
    cout << ref.data;
}
```

Without friend function:

```
void disp(MyClass& ref)
{
    // cannot access private data directly
    // cout << ref.num;  // ❌ Error
    cout << ref.getNum();  // ✅ Access indirectly through getter
}
```

With friend function:

```
class MyClass
{
private:
    int num;
    friend void disp(MyClass&);  // Friend function declaration
public:
    MyClass(int num)
    {
        this->num = num;
    }
};
void disp(MyClass& ref)
{
    cout << ref.num;  // ✅ Direct access (allowed due to friend)
}
```

Important Points:

1. Friend function **is not called using object** (i.e., no object.function()).
2. It can **be called directly** like a normal global function.
3. It **requires an object** as an argument to access its private data.
4. **Friend declaration** can appear **anywhere inside the class** (private/public section).
5. It helps in **operator overloading** where direct access to private members is required.

Friend Function with Multiple Classes

When two classes need to share private data with a **common global function**, that function must be made a **friend of both classes**.

Syntax:

```
class Second; // Forward declaration
class First
{
    int num1;
public:
    First(int num1) { this->num1 = num1; }
    friend int compare(First&, Second&);
};
class Second
{
    int num2;
public:
    Second(int num2) { this->num2 = num2; }
    friend int compare(First&, Second&);
};
```

Definition:

```
int compare(First& f, Second& s)
{
    if (f.num1 > s.num2)
        return 1;
    else if (f.num1 < s.num2)
        return -1;
    else
        return 0;
}
```

Explanation:

- Both First and Second declare compare() as their friend.
- The function can **directly access private data** (num1 and num2) from both classes.
- Used when data comparison or operation involves private data of multiple classes.

Friend Class

A **friend class** is a class that can access **private and protected members** of another class.

Syntax:

```
class B; // forward declaration
class A
{
private:
    int num1;
public:
    A(int n) { num1 = n; }
    friend class B;   // Declaring entire class B as friend
};

class B
{
public:
    void display(A& obj)
    {
        cout << obj.num1;   // ✅ Direct access
    }
};
```

Explanation:

- Here, the entire class B becomes a friend of class A.
- So all member functions of B can directly access private data of A.

Friend Member Function of Another Class

You can also make a **specific member function** of one class a **friend of another class** (not the entire class).

Syntax:

```
class B;
class A
{
private:
    int num1;
public:
    A(int n) { num1 = n; }
    friend void B::disp(A&);  // Only specific member function is friend
};

class B
{
private:
    int num2;
public:
    B(int n) { num2 = n; }
    void disp(A& ref);   // Declaration of member function
};

void B::disp(A& ref)
{
    cout << ref.num1;   // ✅ Allowed due to friendship
}
```

Operator Overloading

Definition:

- Operator Overloading allows using existing operators (like +, -, ==, etc.) with **user-defined data types** (like classes or structures).
- It improves readability and allows class objects to behave similarly to basic data types.

Syntax:

```
return_type operator<symbol>(arguments) {
    // body
}
```

Function Type:

- **Member Function:** Defined inside class.
- **Friend (Global) Function:** Defined outside the class, declared as friend.

---

2. Operators That Cannot Be Overloaded

Operator	Reason
:: (Scope Resolution)	Compile-time binding; determines location of members.
. (Member Access)	Compile-time binding; direct access to class members.
.* (Pointer-to-Member)	Tightly coupled with class internals.
?: (Ternary Operator)	Control flow operator; complex parsing issues.
# (Preprocessor)	Handled by preprocessor, not compiler.

---

3. Understanding Without Operator Overloading

These examples demonstrate how addition (+) can be achieved without overloading the operator.

Example 1 — Simple Member Function

Class Definition:

```
class MyClass {
    int num;
public:
    MyClass(int n) { num = n; }

    void add(MyClass& b) {
        cout << num + b.num << endl;
    }
};
```

In main():

```
MyClass m1(100), m2(200);
m1.add(m2);
```

Explanation:

- Here, addition is done using a **member function**.

- No new object is created.
- Simple addition operation, but syntax is **not intuitive** (uses m1.add(m2) instead of m1 + m2).

Example 2 — Returning Object (Copy Constructor & Destructor Needed)

Class Definition:

```
class MyClass {
    int num;
public:
    MyClass(int n) { num = n; }

    MyClass(const MyClass& ref) {
        this->num = ref.num;
        cout << "Inside copy constructor" << endl;
    }

    MyClass add(MyClass& ref) {
        return num + ref.num;  // temporary object returned
    }
};
```

In main():

```
MyClass m1(100), m2(200);
MyClass m3 = m1.add(m2);
```

Explanation:

- m1.add(m2) returns a **temporary object**.
- While returning, **copy constructor** gets called to copy that temporary object into m3.
- No operator is overloaded yet — this is function-based addition.

Example 3 — Global Friend Function (Without Returning Object)

Class Definition:

```
class MyClass {
    int num;
public:
    MyClass(int n) { num = n; }

    friend void add(MyClass&, MyClass&);
};

void add(MyClass& a, MyClass& b) {
    cout << a.num + b.num << endl;
}
```

In main():

```
MyClass m1(100), m2(200);
add(m1, m2);
```

Explanation:

- Uses a **friend function** for direct access to private members.
- No new object is created.
- Copy constructor and destructor are not needed here.
- Simple functional approach, similar to Example 1 but global.

Example 4 — Global Friend Function Returning Object

Class Definition:

```
class MyClass {
    int num;
public:
    MyClass(int n) { num = n; }

    MyClass(const MyClass& ref) {
        this->num = ref.num;
        cout << "Inside copy constructor" << endl;
    }

    friend MyClass add(MyClass&, MyClass&);
};

MyClass add(MyClass& a, MyClass& b) {
    // Object won't be created here explicitly
    // Returned temporary object will be created in main()
    return MyClass(a.num + b.num);
}
```

In main():

```
MyClass m1(100), m2(200);
MyClass m3 = add(m1, m2);
```

Explanation:

- Temporary object is created and returned.
- **Copy constructor** is called when m3 is initialized.
- This pattern is conceptually equivalent to operator+ overloading.

---

4. When to Write Copy Constructor, Destructor, and Assignment Operator

a) Copy Constructor

You must **explicitly define** a copy constructor when:

1. The class has a **pointer** as a member.
2. Memory is **dynamically allocated** to that pointer.
3. The destructor applies **delete** to that pointer.

Reason:
The compiler-provided copy constructor performs **shallow copy** — copies pointer addresses, not actual data — leading to double deletion or corruption.

Function Type	Binary Operator	Unary Operator
Member Function	1 argument	0 arguments
Friend Function	2 arguments	1 argument

---

1. Printing the Result: m1 + m2;

```
Inside class :
MyClass operator+(MyClass& k)  //m2 goes implicitly
{
    cout << num + k.num << endl;
}
In main :
m1 + m2; //internal representation : m1.operator+(m2) // prints the sum directly
```

Notes: No copy constructor or destructor required (no DMA).

---

2. Unary Operator: m1 + m2;

```
Inside class :
    MyClass operator-()
    {
        // Only one operand (the current object)
        return MyClass(-num);
    }
Inside Main :
    m3 = -m2;    // Internally: m3 = m2.operator-();
```

Notes:
Unary operator - operates on a single object.
Here, m2.operator-() creates a temporary object with negated value

---

3. Returning Object: m1 + m2;

```
Inside class :
    MyClass operator+(MyClass& k) //m2 goes implicitly
{
    return MyClass(num + k.num);              //Anonymous object for return
}
In main :
MyClass m3 = m1 + m2; //m3 = m1.operator+(m2);// returns new object with sum
```

Notes:
Temporary object created — copy constructor may be called.

---

4. Expression: m4 = m2 * m3 + m1;

```
Inside class :
    MyClass operator*(const MyClass& k)       //Method can point anonymous object
{
    return MyClass(num * k.num);
}
MyClass operator+(MyClass& k)
{
    return MyClass(num + k.num);
}
In main :
m4 = m2 + m3 * m1;     //m2 + temp obj //Must follow the order of precedence DMAS
```

Notes:

- Each operator returns a temporary object used in the next operation.
- IN this case reference cannot point to the anonymous object. It can be done using const keyword.

---

5. Compound Assignment: `m1 += 5;`

```
Inside class :
    void operator+=(int val)
{
    num += val;
}
In main :
m1 += 5;        //m1.operator+=(5);
```

Notes:
Updates current object only but doesn't print anything.

---

6. Chained Compound Assignment: `m2 = m1 += 5;`

```
Inside class : (same as above only returning the current object reference : MyClass&)

MyClass& operator+=(int val)
{
    num += val;
    return *this;       // return the current object
                        //this is the pointer which points the current invoking object
                        //* Dereferencing this
}
In main :
m2 = m1 += 5; // first m1 updated, then assigned to m2
```

---

7. Pre-increment: `m2 = ++m1;`

```
Inside class :
    MyClass operator++() // pre-increment
{
    ++num;
    return *this;
}
In main :
m2 = ++m1;              //m2 = m1.operator++()
```

Notes:
Pre-increment modifies and returns *updated* object.

---

8. Post-increment: `m2 = m2++;`

```
Inside class :
    MyClass operator++(int) // post-increment
{
    MyClass temp = *this;
    num++;
    return temp;

        or
   return MyClass(num++);

}
In main :
m2 = m2++;      //m2 = m1.operator++();
```

Notes:
Returns old value before increment.

---

9. Left Operand as Primitive: `m2 = 25 + m1;`

```
Inside class :
    // Inside class declaration
    friend MyClass operator+(int, MyClass&);
Outside class :
    MyClass operator+(int val, MyClass& obj)
{
    return MyClass(val + obj.num);
}
In main :
m2 = 25 + m1;   //m2 = operator+(25, m1);
```

Notes:

- In this scenario the left hand side operand is not object then don't pass any object which can invoke the method implicitly.
- Friend function needed because **first operand (25)** is not an object.

---

10. Function Call and Subscript Operators

```
class MyClass
{
private:
    char* name; // dynamic character array

public:
    // Parameterized constructor
    MyClass(const char* ptr)
    {
        name = new char[strlen(ptr) + 1];
        strcpy_s(name, strlen(ptr) + 1, ptr);
    }

    // Display function
    void disp()
    {
        cout << "Name: " << name << endl;
    }

    // Overloading subscript operator []
    char& operator[](int x)
    {
        if (x >= 0 && x < strlen(name))       //To avoid invalid lengh
        {
            return name[x]; // return character by reference
        }
        else
        {
            cout << "Index out of range!" << endl;
            exit(0);
        }
    }

    // Destructor to release allocated memory
    ~MyClass()
    {
        delete[] name;
        cout << "Memory released successfully.\n";
    }
};

int main()
{
```

```
    MyClass m1("Hello");   // create object with string "Hello"
    m1.disp();

    cout << "First character: " << m1[0]<< endl;      //m1.operator[](0);

    m1[0] = 'M';//m1.operator[](0);  //With  & it will return the reference of character at index

    //Function call
    m1.disp();             // display modified string

    return 0;
}
```

Notes:

- Function call on the left hand side it is possible only when functions returns the reference of object.
- Requires that class has `char arr[SIZE];` or dynamic `char* arr;`.
- If `char* arr` with DMA → must define **copy constructor**, **destructor**, and **operator=** to avoid shallow copy.

---

11. Pointer Access Operator: `m1 -> getnum();`

```
Inside Class :
    // Copy constructor → called when new object is created from existing one
    // Needed for deep copy (to copy actual string, not pointer)
    MyClass(const MyClass& obj) {
        name = new char[strlen(obj.name) + 1];
        strcpy_s(name, strlen(obj.name) + 1, obj.name);
    }

    // Overloaded assignment operator (=)
    // Needed for deep copy during assignment (m4 = m2 = m1)
    MyClass& operator=(const MyClass& obj) {
        if (this != &obj) {                 // avoid self-assignment
            delete[] name;                  // free old memory
            name = new char[strlen(obj.name) + 1];
            strcpy_s(name, strlen(obj.name) + 1, obj.name);
        }
        return *this;                       // return current object (for chaining)

    }

Inside Main :

int main() {
            MyClass m1("Alpha"), m2("Beta"), m4;

            // Here operator= is called twice:
            // 1. m2 = m1; → deep copy Alpha into m2
            // 2. m4 = m2; → deep copy Alpha into m4
            m4 = m2 = m1;

            m1.disp();  // Alpha
            m2.disp();  // Alpha
            m4.disp();  // Alpha
        }
```

When to Overload operator= Explicitly

You must define it when:

- The class contains a pointer member.
- Memory is dynamically allocated on the heap.
- Destructor frees that memory using delete.

Otherwise:

Default assignment operator performs shallow copy → leads to **dangling pointers** or **double-free** errors.

So you must define explicitly:

```
~MyClass();                     // Destructor → to release memory
MyClass(const MyClass&);        // Copy Constructor → for deep copy
MyClass& operator=(const MyClass&); // Assignment Operator → for deep copy on assignment
```

---

12. [] Operator Overloading:

```
Inside class :
    MyClass * operator->()
{
    return this;
}
In main :
m1->getnum();
```

Notes:
Used when object behaves like a pointer.

---

13. Dereference Operator: (*m1).getnum();

```
Inside class :
    MyClass & operator*()
{
    return *this;
}
In main :
(*m1).getnum();
```

Overloading new and delete in C++

You can **overload new and delete operators inside a class** to customize how memory is allocated and released for its objects.
This is mainly used to improve **performance** (e.g., use custom memory pools) or **track memory usage**.

Execution Sequence

Operation	Sequence of Calls
Object creation	1) operator new → 2) Constructor
Object deletion	1) Destructor → 2) operator delete

The overloaded new and delete are **always static** because:

- new is called **before** the object exists (no this pointer yet).
- delete is called **after** the object is destroyed (no members accessible).

Key Points to Remember

1. operator new(size_t size) allocates memory (before constructor).
2. operator delete(void* ptr) frees memory (after destructor).
3. These functions **must** use void* as the return type.
4. Always use malloc() / free() inside custom new and delete if you want full control.
5. For arrays, use operator new[] and operator delete[].

Code Explanation

```
#include <iostream>
#include <cstdlib> // for malloc, free
using namespace std;

class myclass {
private:
    int num;
public:
    // Default constructor
    myclass() {
        num = 0;
        cout << "In default constructor" << endl;
    }

    // Parameterized constructor
    myclass(int num) {
        this->num = num;
        cout << "In parameterized constructor" << endl;
    }

    // Display function
    void disp() {
        cout << "num = " << num << endl;
    }

    // Overloading operator new
    void* operator new(size_t size) {
        cout << "In operator new (allocating " << size << " bytes)" << endl;
        void* ptr = malloc(size); // allocate memory
```

```
        return ptr;
    }
    // Overloading operator new[]
    void* operator new[](size_t size) {
        cout << "In operator new[] (allocating " << size << " bytes)" << endl;
        void* ptr = malloc(size);
        return ptr;
    }
    // Overloading operator delete
    void operator delete(void* ptr) {
        cout << "In operator delete (freeing memory)" << endl;
        free(ptr);
    }
    // Overloading operator delete[]
    void operator delete[](void* ptr) {
        cout << "In operator delete[] (freeing array memory)" << endl;
        free(ptr);
    }
    // Destructor
    ~myclass() {
        cout << "In destructor" << endl;
    }
};
// -------- Main Function --------
int main() {
    // Single object using parameterized constructor
    myclass* m1 = new myclass(10);
    m1->disp();
    delete m1; // calls destructor → operator delete

    cout << "----------------------" << endl;

    // Single object using default constructor
    myclass* m2 = new myclass;
    m2->disp();
    delete m2;

    cout << "----------------------" << endl;

    // Array of objects
    myclass* m3 = new myclass[3]; // calls operator new[] + default constructors
    for (int i = 0; i < 3; i++)
        m3[i].disp();
    delete[] m3; // calls destructors → operator delete[]
}
```

Output Explanation (Typical Flow)

```
In operator new (allocating 4 bytes)
In parameterized constructor
num = 10
In destructor
In operator delete (freeing memory)
---------------------- -
In operator new (allocating 4 bytes)
In default constructor
num = 0
In destructor
In operator delete (freeing memory)
---------------------- -
In operator new[](allocating 12 bytes)
In default constructor
In default constructor
In default constructor
num = 0
num = 0
num = 0
In destructor
In destructor
In destructor
In operator delete[](freeing array memory)
```

Smart Pointers in C++

---

1. Introduction

- A **smart pointer** is an object that behaves like a normal pointer but is **“smart”** because it manages memory automatically (Do more than pointer).
- Can contain advantage of **pointer** as well as **objects.**
- It uses **constructors** and **destructors** to automatically handle memory allocation and deallocation.
- It ensures that dynamically allocated memory is properly released, preventing **memory leaks**, **dangling pointers**, and **manual delete calls**.

---

2. The Need for Smart Pointers

Using raw pointers like this:

```
Person* p = new Person("Scott", 25);
p->display();
delete p;
```

is error-prone because:

- You must manually call delete.
- If an exception occurs or delete is forgotten, memory leaks occur.
- Ownership of memory is unclear in large programs.

Smart pointers solve these problems by automating cleanup.

---

3. Implementing a Simple Smart Pointer (Custom Example)

Code Example

```
#include <iostream>
#include <cstring>
using namespace std;

  // Includes deep copy handling for dynamically allocated memory.

class Person {
    int age;
    char* pName;      // dynamically allocated C-string to store name

public:
    Person() : pName(nullptr), age(0) {}

    // Parameterized constructor
    // Allocates memory for name and copies input string
    Person(const char* pName, int age) {
        this->pName = new char[strlen(pName) + 1];               // allocate memory
        strcpy_s(this->pName, strlen(pName) + 1, pName);         // copy name
        this->age = age;
    }

    // Copy constructor (deep copy)
    // Used when creating a copy of another Person object
    Person(const Person& ref) {
        cout << "Copy constructor called" << endl;
        pName = new char[strlen(ref.pName) + 1];                 // allocate new memory
        strcpy_s(pName, strlen(ref.pName) + 1, ref.pName);       // copy string data
        age = ref.age;                                           // copy age
```

```
    }

    // Destructor
    // Frees dynamically allocated memory
    ~Person() {
        cout << "Person destructor called" << endl;
        delete[] pName;                                          // release heap memory
    }

    // Displays person's name and age
    void display() {
        if (pName)
            cout << pName << "\t" << age << endl;
    }

    // Simple function to demonstrate member access through smart pointer
    void shout() {
        cout << "aaaaaaaaaaaaooooooooooo" << endl;
    }
};

//-----------------------------------------------------------
// Class: SP (Smart Pointer)
// A simple user-defined smart pointer that manages
// a dynamically allocated Person object.
//-----------------------------------------------------------
class SP {
private:
    Person* pData; // Pointer to a Person object that SP will manage

public:
    // Constructor that accepts a raw Person pointer
    explicit SP(Person* pValue) {
        // Create a deep copy of the object using Person's copy constructor
        pData = new Person(*pValue);

        // Delete the original raw pointer since SP now owns the copy
        delete pValue;
    }

    // Destructor automatically called when SP object goes out of scope
    ~SP() {
        cout << "Smart Pointer destructor called" << endl;
        delete pData; // Frees the managed Person object automatically
    }

    // Overload dereference operator (*)
    // Allows SP object to behave like a normal pointer
    Person& operator*() {
        return *pData;
    }

    // Overload arrow operator (->)
    // Enables direct access to Person's members
    Person* operator->() {
        return pData;
    }
};

//-----------------------------------------------------------
// main()
// Demonstrates the behavior of the custom smart pointer SP
//-----------------------------------------------------------
int main() {
    // Create a smart pointer object and initialize it
    // with a dynamically allocated Person
    SP s(new Person("abc", 10));

    // Access Person members using arrow operator
    s->display();
    s->shout();

    // Access Person members using dereference operator
    (*s).display();
    (*s).shout();
```

```
    // When main() ends, the SP destructor is called automatically,
    // which deletes the managed Person object.
    // Hence, no manual delete is needed.
    return 0;
}
```

Explanation

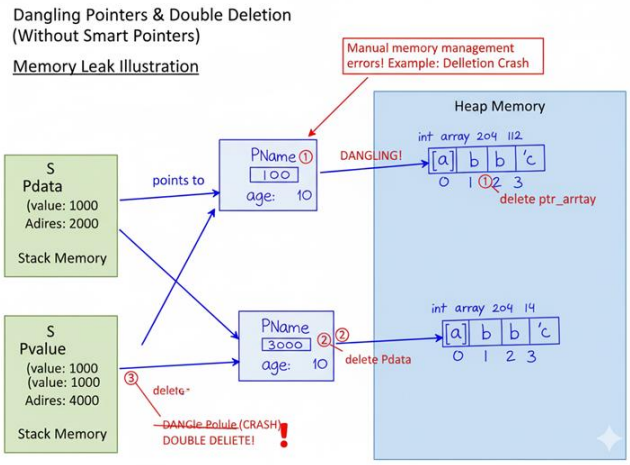
- SP class acts as a **smart pointer** for Person.
- Memory is automatically freed when the smart pointer (s) goes out of scope.
- Overloaded operators -> and * make SP behave like a real pointer.
- Prevents manual memory leaks by using RAII (Resource Acquisition Is Initialization).

---

4. Using Raw Pointers (for Comparison)

```
#include <iostream>
using namespace std;

class Person {
    int age;
    char* pName;

public:
    Person(const char* pName, int age) {
        this->pName = new char[strlen(pName) + 1];
        strcpy_s(this->pName, strlen(pName) + 1, pName);
        this->age = age;
    }

    ~Person() {
        cout << "in destructor" << endl;
        delete[] pName;
    }

    void display() {
        cout << pName << "\t" << age << endl;
    }
};

int main() {
    Person* pPerson = new Person("Scott", 25);
    pPerson->display();
    delete pPerson; // manual deletion required
    return 0;
}
```

Disadvantage: Manual delete is required — if omitted, memory leaks occur.



---

5. Standard Library Smart Pointers

- C++ provides built-in smart pointers in the <memory> header.
- The most commonly used ones are:
  - `std::unique_ptr`
  - `std::shared_ptr`

A. unique_ptr — Sole Ownership

Definition:

- unique_ptr represents **exclusive ownership** of a dynamically allocated object.
- When it goes out of scope, it **automatically deletes** the object.
- It **cannot be copied**, only **moved**.(Cannot call copy constructor)
- `std::unique_ptr` Move only when you want a single owner for a resource.

Example 1: Basic Use

```
#include <iostream>
#include <memory>   // for smart pointers
using namespace std;

// Simple class to demonstrate construction and destruction
class MyClass {
public:
    MyClass() { cout << "MyClass created\n"; }   // Constructor message
    ~MyClass() { cout << "MyClass destroyed\n"; } // Destructor message
};

int main() {
    {
        // Create a unique_ptr that owns a dynamically allocated MyClass object
        unique_ptr<MyClass> ptr1 = make_unique<MyClass>();

        cout << "done\n";  // Indicates object is in use
    } // ptr1 goes out of scope here → object automatically destroyed (no manual delete)

    return 0;
}
```

Key Properties

Feature	Description
Ownership	Only one owner
Copying	❌ Not allowed
Moving	✅ Allowed (using std::move)
Use Case	When a resource must have only one owner

B. shared_ptr — Shared Ownership

Definition:

- shared_ptr allows **multiple pointers** to share ownership of a single dynamically allocated object.
- The object is destroyed **only when the last shared_ptr** goes out of scope.
- `std::unique_ptr` When you want multiple owner for a resource.

Example

```cpp
#include <iostream>
#include <memory>   // required for smart pointers
using namespace std;

// Simple class to show creation and destruction
class MyClass {
public:
    MyClass() { cout << "MyClass created\n"; }   // constructor message
    ~MyClass() { cout << "MyClass destroyed\n"; } // destructor message
};

int main() {
    {
        // Create a unique_ptr that owns a dynamically allocated MyClass object
        unique_ptr<MyClass> ptr1 = make_unique<MyClass>();

        cout << "done\n";  // object is successfully created
    } // ptr1 goes out of scope → destructor called automatically, memory freed

    return 0;
}
```

Key Properties

Feature	Description
Ownership	Shared among multiple pointers
Copying	✅ Allowed
Reference Count	Tracks how many pointers own the object
Destruction	Occurs when count reaches zero
Use Case	When multiple owners need to access the same resource

---

6. Summary Comparison

Type	Ownership	Copyable	Automatically Deletes	Use Case
Raw Pointer	None	Yes	❌ No	Manual memory management
Custom Smart Pointer (SP)	Single	No	✅ Yes	Educational / custom control
unique_ptr	Single	No	✅ Yes	Sole ownership
shared_ptr	Multiple	Yes	✅ Yes	Shared ownership between objects

Type Conversion in C++

Basic Concept of Conversion

In C++, conversion means assigning a value of one data type to another.
The general rule is:

L = R

destination = source

- Conversion direction: **Right → Left (R → L)**.
- If **L and R types are same or L is larger**, conversion is **implicit**.
- If **L is smaller than R**, conversion must be **explicit** (typecasting).

Cases of Implicit and Explicit Conversion

1. **If L and R are of the same type**
2. int = int

→ **Implicit Conversion** (no problem).

3. **If L is larger than R**
4. double = float

→ **Implicit Conversion** (automatic, no data loss).

5. **If L is smaller than R**
6. int = double

→ **Explicit Conversion required** (typecasting needed).

Conversion in Case of User-Defined Data Types (Class or Structure)

When we deal with **user-defined types**, C++ allows three kinds of conversions:

Types of Conversions

1. **Primitive → User Defined**
2. **User Defined → Primitive**
3. **User Defined → User Defined**

---

1. Primitive → User Defined Conversion

This type of conversion is done when a **primitive value** (like int, float, etc.) is assigned to an object of a class.

Possible Methods

a) Using a **Parameterized Constructor** inside the *destination class*.
b) Using **Overloaded Assignment Operator (=)** inside the *destination class*.

Example 1: Using Parameterized Constructor

When you write "m1 = 50", the following happens:

a) A temporary object is created using the parameterized constructor.

b) It is assigned to "m1" using assignment operator (=) provided by the compiler implicitly.

c) The Noname object is destroyed (destructor called).

```
#include<iostream>
using namespace std;

class MyClass
{
private:
    int num;
public:
    MyClass()
    {
        num = 0;
        cout << "In default Const\n";
    }
    /*explicit*/ MyClass(int k)
    {
        cout << endl << "in paramet. cost\n";
        num = k;
    }
    void disp()
    {
        cout << endl << num << endl;
    }
    ~MyClass()
    {
        cout << endl << "In Destructor\t" << num << endl;
    }
};

int main()
{
    MyClass m1(30);
    m1.disp();
    m1 = 50; // possible through parameterized constructor, if constructor is not defined as 'explicit'
    m1.disp();
}
```

Notes:

- Explicit constructor is the one which can be use only for object creation.
- Not for Conversion.

Example 2: Using Assignment Operator (=) with explicit Constructor

```
Inside Class:
    explicit MyClass(int k)
    {
        cout << endl << "in paramet. const\n";
        num = k;
    }

    //operator= overloading

    void operator=(int k)
    {
        this->num = k;
        cout << endl << "in =operator function\n";
    }

int main()
```

```
{
    MyClass m1(30);
    m1.disp();
    MyClass m2;
    m2 = 50; // If constructor is defined as 'explicit', we must overload = operator
    m2.disp();
}
```

---

2. User Defined → Primitive Conversion

- It can haver only one solution
- This is done using a **Conversion Operator Function**.

Conversion Operator Function – Syntax

```
operator <datatype>()
{
    // body
}
```

Rules:

1. It must be a **member function**.
2. It must be written inside the **source class**.
3. It should **not have a return type**, but it must **return a value**.
4. It **cannot take arguments**.
5. We can deal with only one object.

Example: User Defined → Primitive

```
#include<iostream>
using namespace std;

class MyClass
{
private:
    int num;
public:
    MyClass()
    {
        num = 0;
        cout << endl << "In def. const\n";
    }
    MyClass(int k)
    {
        cout << endl << "in paramet. cost\n";
        num = k;
    }
    void disp()
    {
        cout << endl << num << endl;
    }

    //Conversion operator Function
    operator int()
    {
        cout << "inside conversion operator function\t" << num << endl;
        return num;
    }
    ~MyClass()
    {
        cout << endl << "In dest\n";
    }
};
```

```
int main()
{
    MyClass m1(30);
    m1.disp();
    MyClass m2(40);
    m2.disp();
    int val = m2; // invokes conversion operator function
    cout << "val after conversion\n" << val << endl;
    cout << m1 + 300 << endl;  // works because of conversion
    cout << m1 + m2 << endl;   // works because both convert to int
}
```

---

5. User Defined → User Defined Conversion

This can be done in **two ways**:

1. By **overloading operator=** inside the *destination class*.
2. By writing a **conversion operator** inside the *source class*.

Method 1: Operator = Overloaded in Destination

```
#include<iostream>
using namespace std;

class Paise;

class Rupee
{
private:
    int rup;
public:
    Rupee()     { rup = 0;}
    Rupee(int k)       { rup = k;}
    void disp() {cout << endl << "rupee is\t" << rup << endl;}
    void operator=(Paise&);
};

class Paise
{
private:
    int pai;
public:
    Paise()     {pai = 0;}
    Paise(int k)       {pai = k;}
    int getPai()       {return pai;}
    void disp() {cout << endl << "paise  is\t" << pai << endl;}
};

//operator= Overloading
void Rupee::operator=(Paise& k)
{
    this->rup = k.getPai() / 100;
}

int main()
{
    Rupee r(100);
    Paise p(20000);
    r.disp();
    p.disp();
    r = p;     // invokes operator= in destination (Rupee)
    r.disp();
}
```

Method 2: Conversion Operator Inside Source

```
#include<iostream>
using namespace std;

//Destination
class Rupee{};

//Source
class Paise
{
        //Code

    //Conversion Operator Function
    operator Rupee()
    {
        return Rupee(pai / 100);
    }
};

int main()
{
    Rupee r(100);
    Paise p(20000);
    r.disp();
    p.disp();
    r = p;    // invokes conversion operator inside source (Paise)
    r.disp();
}
```

---

6. Summary Table

Conversion Type	Destination	Source
Primitive → UD	Parameterized Constructor Operator=	NA
UD → Primitive	NA	Conversion operator function
UD → UD	Operator=	Conversion operator function
